

Практическая работа №4

Теоретическое введение

Helm — это инструмент управления пакетами для Kubernetes, который упрощает развертывание, обновление и управление приложениями в кластере Kubernetes. Он позволяет автоматизировать процесс установки и настройки приложений, предоставляя возможность определить их конфигурацию в виде чартов.

Чарты Helm представляют собой пакеты, содержащие все необходимые ресурсы Kubernetes для развертывания конкретного приложения или сервиса. Внутри чарта могут быть определены манифесты ресурсов Kubernetes, такие как развертывания, службы, конфигумапы и т. д., а также шаблоны, которые позволяют параметризовать конфигурацию приложения.

В основе Helm лежит концепция шаблонизации, которая позволяет создавать динамические манифесты Kubernetes, подставляя в них переменные и значения из файлов конфигурации. Это позволяет создавать универсальные чарты, которые могут быть переиспользованы в различных средах и на разных кластерах Kubernetes.

Одним из ключевых преимуществ Helm является возможность управления зависимостями между различными чартами. Это позволяет создавать сложные приложения, состоящие из нескольких компонентов, и управлять ими как единым целым.

Кроме того, Helm поддерживает версионирование чартов и релизов, что обеспечивает контроль версий приложений и возможность отката к предыдущим версиям в случае необходимости.

В целом Helm является мощным инструментом для автоматизации развертывания и управления приложениями в Kubernetes, который

упрощает процесс DevOps и улучшает производительность разработчиков и администраторов систем.

Полезные ссылки

1. Установка Minikube: <https://kubernetes.io/ru/docs/tasks/tools/install-minikube/>
2. Установка Kubernetes с помощью Minikube:
<https://kubernetes.io/ru/docs/setup/learning-environment/minikube/>
3. Основы Kubernetes: <https://habr.com/ru/articles/258443/>
4. Managing Workloads:
<https://kubernetes.io/docs/concepts/workloads/management/>
5. Helm | Quickstart Guide: <https://helm.sh/docs/intro/quickstart/>
6. Helm | Chart Development Tips and Tricks:
https://helm.sh/docs/howto/charts_tips_and_tricks/

Задание

Цель работы: установить Helm и с использованием чартов развернуть приложение в кластере.

В данной практической работе необходимо установить Helm, научиться создавать и использовать чарты при развертке в кластере.

Чтобы установить Helm можно использовать различные методы, которые вы можете использовать, ознакомившись с полезными ссылками или документацией Helm. В нашем случае будет использован способ, подходящий для Fedora, а именно команда:

```
sudo dnf install helm
```

Проверить работу установленного Helm можно при помощи команды:

```
helm version
```

Для работы с Helm необходимо создать отдельную директорию, в которой будут находиться такие файлы как chart, values и директория templates.

Давайте подробнее рассмотрим каждый из этих элементов в Helm:

- Chart.yaml:

Назначение: Файл Chart.yaml содержит метаданные чарта, такие как имя, версия, описание и другие детали.

Зачем нужен: Этот файл предоставляет информацию о чарте, которая используется Helm для идентификации и управления версиями чарта. Он также используется при публикации чартов в репозитории.

- values.yaml:

Назначение: Файл values.yaml содержит значения по умолчанию для параметров конфигурации, используемых в шаблонах чарта.

Зачем нужен: Этот файл позволяет определить настройки по умолчанию для вашего приложения. Он предоставляет гибкость для настройки параметров в зависимости от окружения или требований приложения.

- Директория templates:

Назначение: Директория templates содержит шаблоны ресурсов Kubernetes, такие как развертывания, службы, конфигурационные карты и другие.

Зачем нужна: Эта директория является основной частью чарта Helm, так как содержит шаблоны, которые используются для генерации ресурсов Kubernetes. Шаблоны могут быть параметризованы с использованием переменных, определенных в файле values.yaml, что позволяет гибко настраивать конфигурацию приложения.

Иными словами, вы создаете шаблоны манифестов, и изменяете их с помощью chart.yaml и values.yaml.

Вот примеры шаблонов deployment.yaml и service.yaml:

Deployment.yaml

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: {{ include "mcserve-web.fullname" . }}

  labels:

    app: {{ include "mcserve-web.name" . }}

spec:

  replicas: {{ .Values.replicaCount }}

  selector:

    matchLabels:
```

```

    app: {{ include "mcserv-web.name" . }}
template:
  metadata:
    labels:
      app: {{ include "mcserv-web.name" . }}
  spec:
    containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository
}}:{{ .Values.image.tag }}"
        ports:
          - containerPort: 80

```

Service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: {{ include "mcserv-web.fullname" . }}
  labels:
    app: {{ include "mcserv-web.name" . }}
spec:
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: 80

```

```
selector:
```

```
  app: {{ include "mcserve-web.name" . }}
```

По сути, Helm предоставляет возможность настраивать поля ваших манифестов, посредством использования одного файла, передавая данные из него в шаблон.

Вот пример файла Chart.yaml:

```
apiVersion: v2
```

```
name: mcserve-web
```

```
description: A Helm chart for deploying mcserve_web application
```

```
version: 0.1.0
```

Пример файла values.yaml:

```
replicaCount: 1
```

```
image:
```

```
  repository: <ссылка_на_образ_вашего_контейнера>
```

```
  tag: latest
```

```
  pullPolicy: IfNotPresent
```

```
service:
```

```
  type: ClusterIP
```

```
  port: 80
```

Также, возможно, вам придется воспользоваться макросами, создав файл `_helpers.tpl` для верного определения названия параметров в шаблонах. Данный файл должен находиться в папке `templates`.

Пример файла `_helpers.tpl`

```

{{/* Возвращает имя релиза и имени чарта */}}

{{- define "mcserve-web.fullname" -}}

{{- printf "%s-%s" .Release.Name .Chart.Name }}

{{- end }}

{{/* Возвращает имя чарта */}}

{{- define "mcserve-web.name" -}}

{{- default .Chart.Name .Values.nameOverride |
trunc 63 | trimSuffix "-" }}

{{- end }}

```

После создания вашего чарта необходимо его запустить. Сделать это можно, находясь внутри рабочей директории использовав команду:

```
helm install <название_вашего_релиза> .
```

Либо:

```
helm install <название_вашего_релиза>
./<название_рабочей_директории>
```

Если все прошло успешно вы увидите в консоли подобное этому:

```
Release "realise" has been upgraded. Happy
Helming!
```

```
NAME: realise
```

```
LAST DEPLOYED: Mon Apr 22 19:29:01 2024
```

```
NAMESPACE: default
```

```
STATUS: deployed
```

```
REVISION: 3
```



```
TEST SUITE: None
```

Также при использовании команды `kubectl get pods` вы увидите, что развернулся ваш новый под.

Посмотреть список ваших релизов вы можете с помощью команды:

```
helm list
```

С большим количеством команд вы можете ознакомиться в официальной документации Helm в полезных ссылках.

Вопросы к практической работе

1. Что такое Helm и для чего он используется в экосистеме Kubernetes?
2. Какие основные компоненты входят в структуру чарта Helm?
3. Какая роль файлов `Chart.yaml`, `values.yaml` и директории `templates` в чартах Helm?
4. Что такое чарты зависимостей в Helm и как они используются?
5. Как создать новый чарт Helm с помощью команды `helm create`?
6. Как установить чарт Helm в кластер Kubernetes с помощью команды `helm install`?
7. Какие возможности Helm предоставляет для управления версиями чартов и релизов?
8. Как обновить установленный релиз Helm с помощью команды `helm upgrade`?
9. Как удалить установленный релиз Helm с помощью команды `helm uninstall`?
10. Какие лучшие практики следует применять при разработке и использовании чартов Helm?

Критерии оценки

- Установлен Helm
- Создан чарт
- Произведена развертка с помощью чарта
- Составлен отчет о проделанной работе со скриншотами её выполнения
- Даны ответы на вопросы к практической работе